Is Monkeypatching Evil?

In the previous section, you learned that *Monkeypatch* is a derogatory term. However, the same term is sometimes used in a positive sense, to refer to *Open Classes (14)* in general. You might argue that there are two types of *Monkeypatches (16)*. Some happen by mistake, like the one that you and Bill experienced, and they're invariably evil. Others are applied on purpose, and they're quite useful—especially when you want to bend an existing library to your needs.

Even when you think you're in control, you should still Monkeypatch with care. Like any other global modification, Monkeypatches can be difficult to track in a large code base. To minimize the dangers of Monkeypatches, carefully check the existing methods in a class before you define your own methods. Also, be aware that some changes are riskier than others. For example, adding a new method is usually safer than modifying an existing one.

You'll see alternatives to Monkeypatching throughout the book. In particular, we will see soon that you can make Monkeypatches safer by using *Refinements (36)*. Unfortunately, Refinements are still a new feature, and there is no guarantee that they'll ever completely replace traditional Monkeypatches.

Instance Variables

Most importantly, objects contain instance variables. Even though you're not really supposed to peek at them, you can do that anyway by calling `Object#instance_variables`. The object from the previous example has just a single instance variable:

```
obj.my_method
obj.instance_variables  # => [:@v]
```

Unlike in Java or other static languages, in Ruby there is no connection between an object's class and its instance variables. Instance variables just spring into existence when you assign them a value, so you can have objects of the same class that carry different instance variables. For example, if you hadn't called `obj.my_method`, then `obj` would have no instance variable at all. You can think of the names and values of instance variables as keys and values in a hash. Both the keys and the values can be different for each object.

That's all there is to know about instance variables. Let's move on to methods.

Methods

Besides having instance variables, objects also have methods. You can get a list of an object's methods by calling `Object#methods`. Most objects (including `obj` in the previous example) inherit a number of methods from `Object`, so this